

# Lecture Notes

Ian Goodfellow - Chapters 5 to 10

## 1 MACHINE LEARNING BASICS

### The Machine Learning Framework

#### What is Machine Learning?

Machine learning is the study of algorithms that improve their performance at some task with experience.

#### Definition

Three Essential Components of Every ML System:

| Component       | Definition                     | Analogy (Learning Cricket)   |
|-----------------|--------------------------------|------------------------------|
| Task (T)        | What computer should do        | Scoring runs, taking wickets |
| Experience (E)  | Data the algorithm learns from | training sessions            |
| Performance (P) | How we measure success         | Runs scored, wickets taken   |

#### Common Types of Machine Learning Tasks

- **Classification:** Assigning inputs to categories. *Example: Is this email spam or not spam?*
- **Regression:** Predicting a numerical value. *Example: What will be tomorrow's temperature?*
- **Transcription:** Converting unstructured data to text. *Example: Speech recognition, OCR*
- **Translation:** Converting from one language to another. *Example: English to Hindi translation*
- **Anomaly Detection:** Finding unusual patterns. *Example: Credit card fraud detection*
- **Synthesis and Sampling:** Generating new examples similar to training data. *Example: Generating realistic human faces*
- **Imputation:** Filling in missing values. *Example: Predicting missing test results in medical records*
- **Denoising:** Recovering clean data from corrupted input. *Example: Removing noise from images*
- **Density Estimation:** Learning the probability distribution of data. *Example: Understanding where data points are likely to occur*

# Supervised vs Unsupervised Learning

## Supervised Learning

### Definition

Learning from labeled data where each example has an input and a target output.

### Key Characteristics:

- Data has labels/answers
- Learns to map input to output
- Examples: Classification, regression

*Example:* Teaching a child animal names by pointing and saying “This is a dog”

## Unsupervised Learning

### Definition

Learning from unlabeled data to discover patterns and structure.

### Key Characteristics:

- Data has no labels
- Discovers hidden patterns
- Examples: Clustering, dimensionality reduction

*Example:* A child grouping animals by characteristics (furry animals together, feathered animals together) without being told names

## Comparison Table

| Aspect       | Supervised           | Unsupervised             |
|--------------|----------------------|--------------------------|
| Labels       | Yes                  | No                       |
| Goal         | Predict $y$ from $x$ | Discover patterns in $x$ |
| Human effort | High (labeling)      | Low                      |
| Examples     | Spam detection       | Customer segmentation    |

## Capacity, Overfitting, and Underfitting

### Key Definitions

#### Definition

**Generalization:** The ability to perform well on previously unseen inputs.

**Capacity:** The model's ability to fit a wide variety of functions.

**Hypothesis Space:** The set of functions the learning algorithm can choose from.

## Underfitting

### Definition

The model cannot achieve sufficiently low error on the training set.

#### Causes:

- Model too simple (low capacity)
- Not enough training
- Features don't capture important patterns

#### Symptoms:

- High training error
- High test error

## Overfitting

### Definition

The gap between training and test error is too large.

#### Causes:

- Model too complex (high capacity)
- Too many parameters relative to training examples
- Model memorizes noise

#### Symptoms:

- Very low training error
- High test error

## The Capacity-Error Relationship

### Important

Training error decreases with increasing capacity. Test error has a U-shape:

- Low capacity  $\rightarrow$  Underfitting (both errors high)
- Optimal capacity  $\rightarrow$  Best generalization
- High capacity  $\rightarrow$  Overfitting (training low, test high)

## Polynomial Regression Example

- Degree 1: Linear (low capacity)
- Degree 2: Quadratic (medium capacity)
- Degree 9: High-degree polynomial (high capacity, risk of overfitting)

## Representational vs Effective Capacity

- **Representational Capacity**: What functions the model can represent
- **Effective Capacity**: What functions the optimization algorithm can actually find

## Non-parametric Models

Models whose complexity grows with training set size.

*Example:  $k$ -Nearest Neighbors*

- Stores all training examples
- Makes predictions based on nearest neighbors
- No fixed parameter size

## Bayes Error

The minimum possible error, incurred by an oracle that knows the true probability distribution.

## The No Free Lunch Theorem

### Statement

#### Important

When averaged over ALL possible data generating distributions, every classification algorithm has the same error rate on unobserved points.

### Practical Implication

No single algorithm is best for every problem. We must design algorithms based on assumptions about our specific problem.

### What This Means for Practitioners

1. Understand your data's structure
2. Use prior knowledge about the problem
3. Choose algorithms that match your task

# Regularization

## Definition

### Definition

Any modification to a learning algorithm intended to reduce generalization error (not training error).

## Weight Decay (L2 Regularization)

### Formula:

$$J(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda \mathbf{w}^T \mathbf{w}$$

### Where:

- $\lambda$  controls regularization strength
- Larger  $\lambda$  = stronger regularization = smaller weights

### Effect:

- Penalizes large weights
- Encourages simpler models
- Reduces overfitting

## Types of Regularization

| Type              | What it does                      | Effect                           |
|-------------------|-----------------------------------|----------------------------------|
| L2 (Weight Decay) | Penalizes squared weights         | Shrinks weights toward zero      |
| L1 (LASSO)        | Penalizes absolute weights        | Creates sparse weights ( zeros)  |
| Dropout           | Drops units during training       | Forces redundant learning        |
| Early stopping    | Stops training before overfitting | Limits training time             |
| Data augmentation | Creates fake training examples    | Increases effective dataset size |

## Regularization as a Prior

From a Bayesian perspective:

- L2 regularization = Gaussian prior on weights
- L1 regularization = Laplace prior on weights

# Hyperparameters and Validation

## Definitions

### Definition

**Hyperparameters:** Settings that control learning algorithm behavior, not learned from data.

### Examples:

- Learning rate
- Regularization coefficient  $\lambda$
- Number of hidden units
- Polynomial degree

**Parameters:** Learned from data (e.g., weights in linear regression)

## The Validation Set Problem

### Why not use training set for hyperparameter tuning?

- Would always choose maximum capacity  $\rightarrow$  overfitting

### Why not use test set?

- Test set must be kept untouched for final evaluation

## The Solution: Three-Way Split

### Important

Training Set (60-80%)  $\rightarrow$  Learn parameters  
Validation Set (10-20%)  $\rightarrow$  Tune hyperparameters  
Test Set (10-20%)  $\rightarrow$  Final evaluation ONLY

## Cross-Validation

Used when dataset is too small for reliable train/validation split.

### k-Fold Cross-Validation:

1. Split data into k equal parts
2. For  $i = 1$  to k:
  - Use part i as test, remaining k-1 parts as training
3. Average results across all k experiments

## Estimators, Bias, and Variance

### Point Estimation

#### Definition

Providing a single “best” prediction of a quantity of interest.

$$\hat{\theta}_m = g(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)})$$

### Bias

#### Definition

The expected difference between the estimator and the true value.

$$\text{bias}(\hat{\theta}_m) = \mathbb{E}[\hat{\theta}_m] - \theta$$

- **Unbiased estimator:**  $\text{bias} = 0$
- **Asymptotically unbiased:**  $\text{bias} \rightarrow 0$  as  $m \rightarrow \infty$

### Variance

#### Definition

How much the estimator varies across different samples.

$$\text{Var}(\hat{\theta}) = \mathbb{E}[\hat{\theta}^2] - \mathbb{E}[\hat{\theta}]^2$$

### Standard Error

$$\text{SE}(\hat{\theta}) = \sqrt{\text{Var}[\hat{\theta}]}$$

### Mean Squared Error (MSE)

#### Important

$$\text{MSE} = \mathbb{E}[(\hat{\theta}_n - \theta)^2] = \text{Bias}(\hat{\theta}_n)^2 + \text{Var}(\hat{\theta}_n)$$

### Bias-Variance Trade-off

- Simple models: High bias, low variance
- Complex models: Low bias, high variance
- Optimal model balances both

## Important Examples

### Sample Mean (Unbiased):

$$\hat{\mu}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}, \quad \mathbb{E}[\hat{\mu}_m] = \mu$$

### Sample Variance (Biased):

$$\hat{\sigma}_m^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2, \quad \mathbb{E}[\hat{\sigma}_m^2] = \frac{m-1}{m} \sigma^2$$

### Unbiased Sample Variance:

$$\tilde{\sigma}_m^2 = \frac{1}{m-1} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2, \quad \mathbb{E}[\tilde{\sigma}_m^2] = \sigma^2$$

## Consistency

### Definition

As  $m \rightarrow \infty$ , the estimator converges to the true value.

$$\lim_{n \rightarrow \infty} \hat{\theta}_n \xrightarrow{P} \theta$$

**Note:** Asymptotic unbiasedness  $\neq$  Consistency

## Maximum Likelihood Estimation (MLE)

### Principle

#### Important

Choose parameters that make the observed data most probable.

### Formula

$$\theta_{\text{ML}} = \arg \max_{\theta} \prod_{i=1}^m p_{\text{model}}(\mathbf{x}^{(i)}; \theta)$$

### Log-Likelihood (More Convenient)

$$\theta_{\text{ML}} = \arg \max_{\theta} \sum_{i=1}^m \log p_{\text{model}}(\mathbf{x}^{(i)}; \theta)$$

### MLE as Minimizing KL Divergence

Maximizing likelihood = Minimizing KL divergence between empirical distribution and model distribution.

$$D_{\text{KL}}(\hat{p}_{\text{data}} \| p_{\text{model}}) = \mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}} [\log \hat{p}_{\text{data}}(\mathbf{x}) - \log p_{\text{model}}(\mathbf{x})]$$

## Conditional Maximum Likelihood (Supervised Learning)

$$\theta_{\text{ML}} = \arg \max_{\theta} \sum_{i=1}^m \log P(\mathbf{y}^{(i)} | \mathbf{x}^{(i)}; \theta)$$

### MLE for Linear Regression

With Gaussian conditional distribution:

$$\log P(\mathbf{Y} | \mathbf{X}; \theta) = \sum_{i=1}^m \frac{-1}{2\sigma^2} \|\mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)}\|^2 + \text{constant}$$

Maximizing likelihood = Minimizing MSE

### Properties of MLE

| Property    | Description                                           |
|-------------|-------------------------------------------------------|
| Consistency | Converges to true parameter as $m \rightarrow \infty$ |
| Efficiency  | Lowest possible MSE among consistent estimators       |

## Bayesian Statistics

### Key Difference from Frequentist Statistics

| Aspect             | Frequentist       | Bayesian          |
|--------------------|-------------------|-------------------|
| Parameter $\theta$ | Fixed but unknown | Random variable   |
| Data               | Random            | Fixed (observed)  |
| Estimate           | Point estimate    | Full distribution |

### Bayes' Rule

$$p(\theta | \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}) = \frac{p(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)} | \theta)p(\theta)}{p(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)})}$$

- **Prior  $p(\theta)$ :** Belief before seeing data
- **Likelihood:** Evidence from data
- **Posterior:** Updated belief after seeing data

### Prediction with Bayesian Approach

$$p(x^{(m+1)} | x^{(1)}, \dots, x^{(m)}) = \int p(x^{(m+1)} | \theta)p(\theta | x^{(1)}, \dots, x^{(m)})d\theta$$

## Maximum A Posteriori (MAP) Estimation

A point estimate that maximizes posterior probability.

$$\theta_{\text{MAP}} = \arg \max_{\theta} \log p(\mathbf{x} \mid \theta) + \log p(\theta)$$

## MAP = MLE + Regularization

With Gaussian prior, MAP becomes:

$$\theta_{\text{MAP}} = \arg \max_{\theta} \log p(\mathbf{x} \mid \theta) - \lambda \|\theta\|^2$$

This is exactly MLE with L2 regularization!

## Other Important Topics

### Design Matrix

A matrix with examples in rows and features in columns.

*Example:* Iris dataset: 150 examples  $\times$  4 features

### Stochastic Gradient Descent (SGD)

- Uses gradient estimate from minibatch
- Faster convergence than batch gradient descent
- Essential for large datasets

### Curse of Dimensionality

#### Important

Number of possible configurations grows exponentially with dimensions. Need exponentially more data to cover space.

**Solution:** Use assumptions about data structure

### Manifold Hypothesis

Real-world data lies on low-dimensional manifolds embedded in high-dimensional space.

**Implication:** We can learn complex functions by focusing on the manifold where data lives.

### Smoothness Prior (Local Constancy)

Assumption that the function should not change much in small regions.

**Limitation:** With only smoothness prior, need  $O(n)$  examples to distinguish  $O(n)$  regions.

## 2 FEEDFORWARD DEEP NETWORKS

### Architecture Overview

#### Simple Feedforward Network

$$f(\mathbf{x}; \theta) = \mathbf{b} + \mathbf{V} \text{sigmoid}(\mathbf{c} + \mathbf{W}\mathbf{x})$$

##### Components:

- $\mathbf{x}$ : Input vector
- $\mathbf{W}$ : Weight matrix (input to hidden)
- $\mathbf{c}$ : Bias for hidden layer
- $\mathbf{V}$ : Weight matrix (hidden to output)
- $\mathbf{b}$ : Bias for output

#### Hidden Layers

Layers between input and output that learn representations.

**Why they matter:** They transform the data non-linearly, making non-linearly separable problems linearly separable.

#### Universal Approximation Theorem

##### Important

A feedforward network with one hidden layer and enough hidden units can approximate any continuous function to any desired accuracy.

##### Limitations:

- May need exponentially many hidden units
- Optimization may not find the right parameters
- May overfit with too many parameters

#### Why Depth Helps

- Some functions require exponentially fewer units with depth
- Hierarchical feature learning (edges  $\rightarrow$  shapes  $\rightarrow$  parts  $\rightarrow$  objects)
- Better generalization empirically

### Activation Functions

#### Purpose

Add non-linearity to the network. Without non-linearity, stacking layers = single linear transformation.

## Common Activation Functions

| Name       | Formula                                    | Range               | Use Case                |
|------------|--------------------------------------------|---------------------|-------------------------|
| Sigmoid    | $\sigma(a) = \frac{1}{1+e^{-a}}$           | (0,1)               | Binary output           |
| Tanh       | $\tanh(a)$                                 | (-1,1)              | Hidden layers (older)   |
| ReLU       | $\max(\theta, a)$                          | $[\theta, \infty)$  | Hidden layers (default) |
| Leaky ReLU | $\max(\theta, a) + \alpha \min(\theta, a)$ | $(-\infty, \infty)$ | Avoid dead neurons      |
| Softmax    | $\frac{e^{a_i}}{\sum_j e^{a_j}}$           | (0,1), sum=1        | Multi-class output      |

## ReLU Advantages

1. No vanishing gradient for positive inputs
2. Computationally cheap (just max)
3. Sparse activations
4. Excellent empirical performance

## Dead ReLU Problem

If weights push a neuron to be always negative, it never activates.

**Solutions:**

- Leaky ReLU (small slope for negatives)
- Proper initialization

## Loss Functions

### Maximum Likelihood Principle for Loss

Instead of designing loss functions, use negative log-likelihood:

$$L(\mathbf{x}, \mathbf{y}) = -\log p(\mathbf{y} \mid \mathbf{x})$$

### Binary Classification Loss

**Model:**  $p(y = 1 \mid x) = \sigma(a)$

**Binary Cross-Entropy:**

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

### Multi-Class Classification Loss

**Model:**  $\mathbf{p} = \text{softmax}(\mathbf{a})$

**Categorical Cross-Entropy:**

$$L = -\log(p_{\text{correct}})$$

## Regression Loss

**Model:**  $p(y | x) = \mathcal{N}(\mu = f(x), \sigma^2)$

**Mean Squared Error:**

$$L = \frac{1}{2\sigma^2}(y - \mu)^2 + \text{constant}$$

## Why Cross-Entropy Over Squared Error for Classification?

With sigmoid/softmax, squared error has vanishing gradient when output is confidently wrong. Cross-entropy maintains strong gradient.

## Softmax Gradient

$$\frac{\partial L}{\partial a_k} = p_k - \mathbf{1}_{k=y}$$

- Correct class: pushed up (negative gradient)
- Wrong classes: pushed down (positive gradient)

## Back-Propagation

### Definition

Efficient algorithm for computing gradients of loss with respect to all parameters.

### The Chain Rule

**Scalar case:**

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$$

**Vector case:**

$$\nabla_{\theta} J(g(\theta)) = \nabla_{g(\theta)} J(g(\theta)) \frac{\partial g(\theta)}{\partial \theta}$$

## Forward Propagation (Making Predictions)

For each layer  $k$  from 1 to  $l$ :

$$\begin{aligned}\mathbf{a}^{(k)} &= \mathbf{b}^{(k)} + \mathbf{W}^{(k)} \mathbf{h}^{(k-1)} \\ \mathbf{h}^{(k)} &= f(\mathbf{a}^{(k)})\end{aligned}$$

Where  $\mathbf{h}^{(0)} = \mathbf{x}$  and output  $\hat{\mathbf{y}} = \mathbf{h}^{(l)}$

## Backward Propagation (Computing Gradients)

Start with gradient on output:

$$\mathbf{g} = \nabla_{\hat{\mathbf{y}}} J$$

For each layer  $k$  from  $l$  down to 1:

1. Gradient through activation:

$$\mathbf{g} = \nabla_{\mathbf{a}^{(k)}} J = \mathbf{g} \odot f'(\mathbf{a}^{(k)})$$

2. Gradients for parameters:

$$\begin{aligned}\nabla_{\mathbf{b}^{(k)}} J &= \mathbf{g} \\ \nabla_{\mathbf{W}^{(k)}} J &= \mathbf{g} \mathbf{h}^{(k-1)\top}\end{aligned}$$

3. Propagate to previous layer:

$$\mathbf{g} = \nabla_{\mathbf{h}^{(k-1)}} J = \mathbf{W}^{(k)\top} \mathbf{g}$$

## fprop and bprop

- **fprop (forward propagation)**: Computes output, stores intermediate values
- **bprop (backward propagation)**: Computes gradients using stored values

## Why Back-Propagation is Efficient

Naive approach: Perturb each weight individually  $\rightarrow$   $O(\text{parameters})$  forward passes

Back-propagation: One forward + one backward pass  $\rightarrow O(1)$  passes

## Computational Graph

Nodes represent values, edges represent operations. Allows automatic differentiation.

## Reparameterization Trick

### Problem

Sampling operations are not differentiable. Cannot back-propagate through random sampling.

### Solution

Rewrite sampling as deterministic function + independent noise.

**Gaussian Example:**

$$y \sim \mathcal{N}(\mu, \sigma^2) \quad \Rightarrow \quad y = \mu + \sigma\eta, \quad \eta \sim \mathcal{N}(0, 1)$$

Now gradients can flow to  $\mu$  and  $\sigma$ .

## REINFORCE Algorithm

For discrete stochastic operations:

$$\frac{\partial \mathbb{E}[J(y)]}{\partial \omega} \approx \frac{1}{n} \sum_{i=1}^n J(y^{(i)}) \frac{\partial \log p(y^{(i)})}{\partial \omega}$$

## Piecewise Linear Units

### Historical Context

Sigmoid and tanh (saturating) were used historically but have vanishing gradient problems.

| Unit       | Formula                          | Key Property                |
|------------|----------------------------------|-----------------------------|
| ReLU       | $\max(0, a)$                     | Gradient 1 for active units |
| Leaky ReLU | $\max(0, a) + 0.01 \min(0, a)$   | Small slope when negative   |
| PReLU      | $\max(0, a) + \alpha \min(0, a)$ | $\alpha$ learned from data  |

## ReLU Family

### Maxout Units

$$h_i = \max_{j \in G_i} (b_{ij} + \mathbf{w}_{ij}^\top \mathbf{x})$$

- Generalizes ReLU
- Multiple weight vectors per unit
- Never has zero gradient
- Can approximate any convex function

### Why Piecewise Linear Units Work

1. No vanishing gradients (slopes are 0 or 1, not tiny)
2. Sparse activations (many zeros)
3. Fast computation
4. Good gradient flow through back-propagation

## 3 REGULARIZATION

### Introduction to Regularization

#### Definition

##### Definition

Any modification to a learning algorithm intended to reduce generalization error (not training error).

#### Bayesian Perspective

Regularization = MAP estimation with prior

- L2 regularization  $\rightarrow$  Gaussian prior on weights
- L1 regularization  $\rightarrow$  Laplace prior on weights

### L2 Regularization (Weight Decay)

#### Formula

$$\tilde{J}(\mathbf{w}) = J(\mathbf{w}) + \frac{\alpha}{2} \|\mathbf{w}\|_2^2$$

#### Effect

- Shrinks weights toward zero
- Reduces variance (at cost of increased bias)
- Equivalent to Gaussian prior

#### Linear Regression with L2

Normal equations become:

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \alpha \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

### L1 Regularization

#### Formula

$$\tilde{J}(\mathbf{w}) = J(\mathbf{w}) + \alpha \|\mathbf{w}\|_1 = J(\mathbf{w}) + \alpha \sum_i |w_i|$$

#### Effect

- Creates sparse solutions (many weights become exactly zero)
- Useful for feature selection
- Equivalent to Laplace prior

## Why L1 Produces Sparsity

Gradient update: constant magnitude regardless of weight size. Forces weights to zero when they're small.

## Comparison of L1 and L2

| Aspect          | L2          | L1                      |
|-----------------|-------------|-------------------------|
| Penalty         | $\ w\ ^2$   | $\ w\ $                 |
| Solution        | Dense       | Sparse                  |
| Gradient        | Linear (2w) | Constant (sign(w))      |
| Bayesian prior  | Gaussian    | Laplace                 |
| Computationally | Smooth      | Not differentiable at 0 |

## Dropout

### Definition

Randomly dropping units (along with their connections) during training with probability  $p$ .

### How It Works

- Each minibatch uses a different subset of the network
- At test time, use all units with weights scaled by dropout probability

### Why It Works

- Forces network to learn redundant representations
- Prevents co-adaptation of neurons
- Approximates training an ensemble of many subnetworks

### Weight Scaling Inference Rule

At test time, multiply weights by dropout probability  $p$ .

## Early Stopping

### Definition

Stop training when validation error starts increasing.

## Algorithm

1. Train on training set
2. Periodically evaluate on validation set
3. Keep best parameters (lowest validation error)
4. Stop when validation error hasn't improved for patience steps

## Why It Works

- Limits the number of training steps
- Equivalent to L2 regularization (for quadratic objectives)
- Most commonly used regularization in deep learning

## Dataset Augmentation

### Definition

Creating fake training data by applying transformations.

### Common Transformations for Images

- Random crops
- Horizontal flips
- Rotations
- Color jitter
- Scaling

## Why It Works

- Increases effective dataset size
- Makes model invariant to transformations
- Reduces overfitting

## Noise Robustness

### Adding Noise to Inputs

- Equivalent to penalizing the norm of the gradient
- Makes model locally robust

## **Adding Noise to Weights**

- Encourages model to be insensitive to weight perturbations
- Useful for recurrent networks

## **Parameter Sharing**

### **Definition**

Using the same parameter for multiple functions in the model.

### **Example: Convolutional Networks**

Same kernel used at all spatial locations.

### **Benefits**

- Reduces number of parameters
- Imposes translation equivariance prior
- Better statistical efficiency

## **Sparse Representations**

### **Definition**

Regularizing the representation (output of hidden layers), not parameters.

### **L1 Penalty on Hidden Units**

- Encourages sparse hidden activations
- Many hidden units output zero

### **Benefits**

- More interpretable
- Efficient storage and computation

## **Bagging and Ensemble Methods**

### **Bagging (Bootstrap Aggregating)**

- Train  $k$  models on different bootstrap samples of data
- Average predictions at test time

## **Why Ensembles Work**

- Different models make different errors
- Averaging reduces variance
- Expected MSE improves if errors are uncorrelated

## **Multi-Task Learning**

### **Definition**

Sharing parameters across multiple related tasks.

### **Architecture**

- Shared lower layers (learn common features)
- Task-specific upper layers

### **Benefits**

- More data for shared parameters
- Better generalization if tasks are related
- Prior that tasks share common factors

## **Adversarial Training**

### **Adversarial Examples**

Small, imperceptible perturbations that cause model to misclassify.

### **Cause**

Excessive linearity in neural networks.

### **Adversarial Training**

Train on adversarial examples to improve robustness.

## 4 OPTIMIZATION FOR TRAINING DEEP MODELS

### Optimization vs Generalization

#### Empirical Risk Minimization

Minimize loss on training set.

#### Surrogate Loss Functions

Use differentiable loss (e.g., cross-entropy) even when we care about non-differentiable loss (e.g., classification error).

#### Batch vs Minibatch vs Online

| Type      | Uses           | Update frequency         |
|-----------|----------------|--------------------------|
| Batch     | Full dataset   | Once per epoch           |
| Minibatch | Subset         | Multiple times per epoch |
| Online    | Single example | Every example            |

### Challenges in Neural Network Optimization

#### Ill-Conditioning

- Hessian matrix has very different eigenvalues
- Gradient descent bounces in high-curvature directions
- Need very small learning rates

#### Local Minima

- For large networks, most local minima have low cost
- Non-identifiability (weight space symmetry) creates many equivalent minima
- Not the main problem in practice

#### Saddle Points

- Points where gradient is zero but not a minimum
- Have both positive and negative eigenvalues
- Far more common than local minima in high dimensions
- Gradient descent is repelled from saddle points

## Cliffs and Exploding Gradients

- Sharp regions where gradient becomes huge
- Common in RNNs (recurrent weight multiplication)
- Solution: Gradient clipping

## Vanishing Gradients

- Gradients become exponentially small with depth/time
- Common in deep networks and RNNs
- Solutions: ReLU, LSTM, skip connections, careful initialization

## Long-Term Dependencies (RNNs)

- To store information, network must be contractive
- Contractive dynamics  $\rightarrow$  vanishing gradients
- Fundamental trade-off

## Gradient Clipping

### Purpose

Prevent exploding gradients.

### Norm Clipping

$$\text{if } \|\mathbf{g}\| > v, \quad \mathbf{g} \leftarrow \frac{v}{\|\mathbf{g}\|} \mathbf{g}$$

### Element-wise Clipping

Cap each gradient element to a maximum value.

## Basic Optimization Algorithms

### Gradient Descent (Batch)

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$$

### Stochastic Gradient Descent (SGD)

$$\theta \leftarrow \theta - \eta \hat{\nabla}_{\theta} J(\theta) \quad (\text{from minibatch})$$

**Learning rate schedule:** Must decay to zero for convergence.

## Momentum

### Standard Momentum

$$\begin{aligned}\mathbf{v} &\leftarrow \alpha \mathbf{v} - \eta \nabla_{\theta} J(\theta) \\ \theta &\leftarrow \theta + \mathbf{v}\end{aligned}$$

- $\alpha$  is momentum coefficient (typically 0.9)
- Accelerates in directions of consistent gradient
- Helps overcome small gradients

### Nesterov Momentum

$$\begin{aligned}\theta_{\text{lookahead}} &= \theta + \alpha \mathbf{v} \\ \mathbf{v} &\leftarrow \alpha \mathbf{v} - \eta \nabla_{\theta} J(\theta_{\text{lookahead}}) \\ \theta &\leftarrow \theta + \mathbf{v}\end{aligned}$$

- Evaluates gradient after applying momentum
- Better convergence rate for convex problems

## Adaptive Learning Rate Methods

### AdaGrad

$$\begin{aligned}\mathbf{r} &\leftarrow \mathbf{r} + \mathbf{g}^2 \\ \theta &\leftarrow \theta - \frac{\eta}{\sqrt{\mathbf{r} + \epsilon}} \mathbf{g}\end{aligned}$$

- Adapts learning rates per parameter
- Larger updates for infrequent parameters
- Accumulates squared gradients (never forgets)

### RMSprop

$$\begin{aligned}\mathbf{r} &\leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g}^2 \\ \theta &\leftarrow \theta - \frac{\eta}{\sqrt{\mathbf{r} + \epsilon}} \mathbf{g}\end{aligned}$$

- Uses moving average instead of accumulation
- Addresses AdaGrad's diminishing learning rates

## Adam (Adaptive Moment Estimation)

$$\begin{aligned}\mathbf{s} &\leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g} && \text{(first moment)} \\ \mathbf{r} &\leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g}^2 && \text{(second moment)} \\ \hat{\mathbf{s}} &= \frac{\mathbf{s}}{1 - \rho_1^t}, \quad \hat{\mathbf{r}} = \frac{\mathbf{r}}{1 - \rho_2^t} \\ \theta &\leftarrow \theta - \eta \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \epsilon}}\end{aligned}$$

- Combines momentum with RMSprop
- Bias corrections for early steps
- Most popular optimizer for deep learning

## Parameter Initialization

### Importance

- Breaks symmetry between units
- Affects convergence speed and final performance
- Too small  $\rightarrow$  vanishing signals
- Too large  $\rightarrow$  exploding signals/saturation

### Common Heuristics

#### Xavier/Glorot Initialization:

$$W_{ij} \sim U \left( -\frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}}, \frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}} \right)$$

#### He Initialization (for ReLU):

$$W_{ij} \sim \mathcal{N} \left( 0, \frac{2}{n_{\text{in}}} \right)$$

### Bias Initialization

- Usually set to 0
- Sometimes set to small positive number (e.g., 0.1 for ReLU)
- For output layers, can set to match marginal statistics

## Pretraining

### Supervised Pretraining

- Train on related task with more data
- Transfer learned features to target task

## **Unsupervised Pretraining**

- Train layer-by-layer using unsupervised learning
- Then fine-tune with supervised learning

## **Why Pretraining Helps**

- Better initialization
- Encodes prior knowledge
- Helps optimization

## **Continuation Methods**

### **Definition**

Solve a series of increasingly difficult problems.

### **Curriculum Learning**

Present training examples in a meaningful order (easy to hard).

## 5 CONVOLUTIONAL NETWORKS

### The Convolution Operation

#### Definition

Continuous convolution:

$$s(t) = (x * w)(t) = \int x(a)w(t - a)da$$

Discrete convolution:

$$s[t] = (x * w)[t] = \sum_{a=-\infty}^{\infty} x[a]w[t - a]$$

2D convolution:

$$S[i, j] = (I * K)[i, j] = \sum_m \sum_n I[i - m, j - n]K[m, n]$$

#### Terminology

- **Input (x, I):** The data being processed
- **Kernel (w, K):** The learnable filter
- **Feature map (s, S):** The output of convolution

### Key Properties of Convolution

#### Sparse Connectivity

- Each output connects to only a small region of input
- Kernel size  $\ll$  input size
- Reduces number of parameters

#### Parameter Sharing

- Same kernel used at all positions
- Dramatically reduces parameters
- Imposes translation equivariance prior

#### Equivariance

- Convolution is equivariant to translation
- If input shifts, output shifts by same amount
- Important for image processing

## Convolutional Layer Variants

### Multi-Channel Convolution

Input has multiple channels (e.g., RGB images).

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m,k+n} K_{i,l,m,n}$$

Where:

- i: output channel
- l: input channel
- j,k: spatial positions
- m,n: kernel offsets

### Strided Convolution

Skip some positions when sliding kernel.

- Reduces output size
- Computational savings

### Padding

- **Valid:** No padding → output smaller
- **Same:** Pad to keep output size same as input
- **Full:** Pad enough to compute convolution at all positions

## Pooling

### Purpose

- Reduce spatial dimensions
- Provide translation invariance
- Increase receptive field

### Max Pooling

Output = maximum value in pooling window

$$\text{max-pool}(I)_{i,j} = \max_{m,n \in \text{window}} I[i+m, j+n]$$

### Average Pooling

Output = average of values in pooling window

## Why Pooling Provides Invariance

Small translations cause values to shift within pooling window, but max pooling still detects presence of feature.

## Convolution as an Infinitely Strong Prior

### Interpretation

A fully connected net with an infinitely strong prior that:

- Weights are zero outside local receptive fields
- Weights are identical across spatial positions

### Implications

- Good when translation equivariance is appropriate
- Can underfit if task requires precise spatial information

## CNN Architecture Design

### Typical Structure

Input Image

↓

[Convolution → Activation → Pooling] (repeat several times)

↓

Flatten

↓

[Fully Connected → Activation] (1-2 times)

↓

Output (Softmax)

### Why This Works

- Early layers: Detect simple features (edges, colors)
- Middle layers: Detect parts (eyes, wheels)
- Later layers: Detect whole objects

## Efficient Convolution Algorithms

### Fourier Transform

Convolution in spatial domain = point-wise multiplication in frequency domain.

### Separable Convolution

If kernel = outer product of vectors, can use 1D convolutions.

## **Im2Col**

Convert convolution to matrix multiplication.

## **Random vs Unsupervised Features**

### **Random Filters**

- Surprisingly effective for CNNs
- Natural images have structure that random filters capture

### **Unsupervised Pretraining for CNNs**

- Train first layer with unsupervised learning
- Extract features only once
- Train classifier on features

## **Neuroscience Connection**

### **V1 (Primary Visual Cortex)**

- **Simple cells:** Detect edges at specific orientations (like convolution filters)
- **Complex cells:** Invariant to small translations (like max pooling)

### **Learned Features**

First-layer filters in CNNs trained on natural images resemble Gabor functions (found in V1).

## 6 RECURRENT NEURAL NETWORKS

### Unfolding Computational Graphs

#### Dynamical System

$$\mathbf{s}^{(t)} = f(\mathbf{s}^{(t-1)}; \theta)$$

#### Recurrent Network with Input

$$\mathbf{s}^{(t)} = f(\mathbf{s}^{(t-1)}, \mathbf{x}^{(t)}; \theta)$$

#### Unfolding

Convert recurrent circuit into computational graph with repeated structure.

- Same parameters shared across time steps
- Handles variable-length sequences

### Basic RNN Architecture

#### Forward Propagation Equations

$$\mathbf{a}^{(t)} = \mathbf{b} + \mathbf{W}\mathbf{h}^{(t-1)} + \mathbf{U}\mathbf{x}^{(t)}$$

$$\mathbf{h}^{(t)} = \tanh(\mathbf{a}^{(t)})$$

$$\mathbf{o}^{(t)} = \mathbf{c} + \mathbf{V}\mathbf{h}^{(t)}$$

$$\hat{\mathbf{y}}^{(t)} = \text{softmax}(\mathbf{o}^{(t)})$$

Where:

- $\mathbf{x}^{(t)}$ : Input at time t
- $\mathbf{h}^{(t)}$ : Hidden state at time t
- $\mathbf{U}$ : Input-to-hidden weights
- $\mathbf{W}$ : Hidden-to-hidden weights
- $\mathbf{V}$ : Hidden-to-output weights
- $\mathbf{b}, \mathbf{c}$ : Biases

#### Loss

$$L = \sum_{t=1}^T L^{(t)} = - \sum_{t=1}^T \log p(y^{(t)} | \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(t)})$$

### Back-Propagation Through Time (BPTT)

#### Key Idea

Unfold RNN through time, then apply standard back-propagation.

## Gradient for Hidden State

$$\nabla_{\mathbf{h}^{(t)}} L = (\nabla_{\mathbf{h}^{(t+1)}} L) \frac{\partial \mathbf{h}^{(t+1)}}{\partial \mathbf{h}^{(t)}} + (\nabla_{\mathbf{o}^{(t)}} L) \frac{\partial \mathbf{o}^{(t)}}{\partial \mathbf{h}^{(t)}}$$

## Vanishing/Exploding Gradients in RNNs

$$\frac{\partial \mathbf{h}^{(T)}}{\partial \mathbf{h}^{(t)}} = \prod_{k=t}^{T-1} \frac{\partial \mathbf{h}^{(k+1)}}{\partial \mathbf{h}^{(k)}}$$

- Product of many Jacobians
- If eigenvalues  $\geq 1 \rightarrow$  explode
- If eigenvalues  $\leq 1 \rightarrow$  vanish

## Spectral Radius

Maximum absolute eigenvalue of the Jacobian matrix.

- $\geq 1$ : Exploding gradients
- $\leq 1$ : Vanishing gradients
- $= 1$ : Stable information flow

## Teacher Forcing

### Definition

During training, feed true previous output (not model's prediction) as input.

### Advantage

- Stabilizes training
- Decouples time steps

### Disadvantage

Train/test mismatch (exposure bias)

### Solution

Curriculum learning: Gradually transition from teacher forcing to free-running.

## Bidirectional RNNs

### Architecture

- Forward RNN processes sequence left to right
- Backward RNN processes sequence right to left
- Concatenate states for output

## Why Useful

Many tasks require context from both past and future (e.g., speech recognition, handwriting recognition).

## Encoder-Decoder (Sequence-to-Sequence)

### Architecture

#### Encoder RNN:

- Reads input sequence
- Produces context vector  $C$  (last hidden state)

#### Decoder RNN:

- Conditioned on context  $C$
- Generates output sequence

### Context Vector

Fixed-size representation of entire input sequence.

### Limitation

Fixed-size context may not capture long sequences well.

## Attention Mechanism

### Problem with Basic Encoder-Decoder

Fixed context vector bottlenecks information flow for long sequences.

### Solution

At each decoder step, attend to different parts of encoder outputs.

$$\text{attention weights} = \text{softmax}(\text{score}(h_{\text{dec}}, h_{\text{enc}}))$$

$$\text{context vector} = \sum \text{attention weights} \times h_{\text{enc}}$$

### Benefits

- Handles long sequences better
- More interpretable (see where model is looking)
- Improved performance on translation, captioning

## LSTM (Long Short-Term Memory)

### Why LSTM

Addresses vanishing gradient problem in RNNs.

### Cell State

$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \odot \tilde{\mathbf{c}}^{(t)}$$

- Linear self-loop allows gradients to flow
- Forget gate controls how much to remember

### Gates

**Forget Gate:** How much to forget previous state

$$\mathbf{f}^{(t)} = \sigma(\mathbf{b}_f + \mathbf{U}_f \mathbf{x}^{(t)} + \mathbf{W}_f \mathbf{h}^{(t-1)})$$

**Input Gate:** How much to add new information

$$\mathbf{i}^{(t)} = \sigma(\mathbf{b}_i + \mathbf{U}_i \mathbf{x}^{(t)} + \mathbf{W}_i \mathbf{h}^{(t-1)})$$

**Candidate Cell State:**

$$\tilde{\mathbf{c}}^{(t)} = \tanh(\mathbf{b}_c + \mathbf{U}_c \mathbf{x}^{(t)} + \mathbf{W}_c \mathbf{h}^{(t-1)})$$

**Output Gate:** How much to output

$$\mathbf{o}^{(t)} = \sigma(\mathbf{b}_o + \mathbf{U}_o \mathbf{x}^{(t)} + \mathbf{W}_o \mathbf{h}^{(t-1)})$$

**Hidden State:**

$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \odot \tanh(\mathbf{c}^{(t)})$$

### Why LSTM Works

- Constant error carousel: gradients flow through cell state with little decay
- Gating mechanisms control information flow dynamically
- Can learn long-term dependencies (hundreds of steps)

## Gated Recurrent Unit (GRU)

### Simplified LSTM

- Combines forget and input gates into update gate
- No separate cell state

### Update Gate

$$\mathbf{u}^{(t)} = \sigma(\mathbf{b}_u + \mathbf{U}_u \mathbf{x}^{(t)} + \mathbf{W}_u \mathbf{h}^{(t-1)})$$

## Reset Gate

$$\mathbf{r}^{(t)} = \sigma(\mathbf{b}_r + \mathbf{U}_r \mathbf{x}^{(t)} + \mathbf{W}_r \mathbf{h}^{(t-1)})$$

## Candidate Hidden State

$$\tilde{\mathbf{h}}^{(t)} = \tanh(\mathbf{b}_h + \mathbf{U}_h \mathbf{x}^{(t)} + \mathbf{W}_h (\mathbf{r}^{(t)} \odot \mathbf{h}^{(t-1)}))$$

## Final Hidden State

$$\mathbf{h}^{(t)} = \mathbf{u}^{(t)} \odot \mathbf{h}^{(t-1)} + (1 - \mathbf{u}^{(t)}) \odot \tilde{\mathbf{h}}^{(t)}$$

## Deep RNNs

### Architecture Options

1. **Stacked RNNs:** Multiple recurrent layers
2. **Deep transitions:** MLP for state-to-state mapping
3. **Skip connections:** Shortcut paths to help gradient flow

### Benefits

- More capacity
- Hierarchical feature learning over time

## Gradient Clipping for RNNs

### Purpose

Prevent exploding gradients in RNNs.

### Norm Clipping

$$\text{if } \|g\| > \text{threshold, } g \leftarrow \frac{\text{threshold}}{\|g\|} g$$

### Why Important for RNNs

Recurrent multiplication of weight matrices can cause exponential growth.

## Managing Long-Term Dependencies

1. **LSTM/GRU:** Gating mechanisms
2. **Gradient clipping:** Handle explosions
3. **Regularization to encourage information flow:** Penalize vanishing gradients
4. **Skip connections:** Short paths through time
5. **Multiple time scales:** Different groups of units at different rates

## 7 Important Formulas

| Concept                       | Formula                                                                                                          |
|-------------------------------|------------------------------------------------------------------------------------------------------------------|
| Linear Regression             | $\hat{y} = \mathbf{w}^\top \mathbf{x}$                                                                           |
| MSE                           | $\frac{1}{m} \sum (\hat{y} - y)^2$                                                                               |
| Bias                          | $\mathbb{E}[\hat{\theta}] - \theta$                                                                              |
| Variance                      | $\mathbb{E}[\hat{\theta}^2] - \mathbb{E}[\hat{\theta}]^2$                                                        |
| MSE = Bias <sup>2</sup> + Var | $\mathbb{E}[(\hat{\theta} - \theta)^2] = \text{Bias}^2 + \text{Var}$                                             |
| MLE                           | $\arg \max_{\theta} \prod p(x^{(i)}; \theta)$                                                                    |
| MAP                           | $\arg \max_{\theta} [\log p(x \theta) + \log p(\theta)]$                                                         |
| L2 Regularization             | $J(\mathbf{w}) + \lambda \ \mathbf{w}\ _2^2$                                                                     |
| L1 Regularization             | $J(\mathbf{w}) + \lambda \ \mathbf{w}\ _1$                                                                       |
| Sigmoid                       | $\sigma(a) = 1/(1 + e^{-a})$                                                                                     |
| ReLU                          | $\max(0, a)$                                                                                                     |
| Softmax                       | $e^{a_i} / \sum_j e^{a_j}$                                                                                       |
| Chain Rule                    | $\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x}$                    |
| RNN State                     | $\mathbf{h}^{(t)} = \tanh(\mathbf{b} + \mathbf{W}\mathbf{h}^{(t-1)} + \mathbf{U}\mathbf{x}^{(t)})$               |
| LSTM Cell                     | $\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \odot \tilde{\mathbf{c}}^{(t)}$ |
| Convolution                   | $(I * K)[i, j] = \sum_m \sum_n I[m, n] K[i - m, j - n]$                                                          |
| Momentum                      | $\mathbf{v} \leftarrow \alpha \mathbf{v} - \eta \mathbf{g}, \quad \theta \leftarrow \theta + \mathbf{v}$         |

## 8 Key Takeaways

### Chapter 5

1. Every ML system has Task, Experience, Performance
2. Supervised = labeled data, Unsupervised = no labels
3. Underfitting = model too simple, Overfitting = model too complex
4. Regularization reduces overfitting (L1, L2, dropout, early stopping)
5. Bias = systematic error, Variance = inconsistency
6.  $MSE = Bias^2 + Variance$
7. MLE chooses parameters maximizing data likelihood
8. MAP adds prior (regularization) to MLE

### Chapter 6

1. Hidden layers learn representations
2. ReLU is default activation (no vanishing gradient)
3. Cross-entropy for classification, MSE for regression
4. Back-propagation efficiently computes gradients using chain rule
5. Universal approximation: one hidden layer can learn any function
6. Depth helps learn functions more efficiently

### Chapter 7

1. L2 shrinks weights, L1 creates sparsity
2. Dropout trains ensemble of subnetworks
3. Early stopping = regularization by limiting training time
4. Data augmentation creates fake training examples
5. Parameter sharing imposes priors (e.g., translation equivariance)

### Chapter 8

1. Optimization challenges: ill-conditioning, saddle points, vanishing/exploding gradients
2. Momentum accelerates convergence
3. Adam = momentum + RMSprop (most popular optimizer)
4. Gradient clipping prevents exploding gradients
5. Proper initialization crucial for training

## Chapter 9

1. Convolution = sparse connectivity + parameter sharing
2. Provides translation equivariance
3. Pooling provides translation invariance
4. CNNs learn hierarchical features (edges  $\rightarrow$  parts  $\rightarrow$  objects)

## Chapter 10

1. RNNs process sequences by sharing parameters across time
2. BPTT computes gradients, but suffers from vanishing/exploding gradients
3. LSTM uses gating mechanisms to preserve gradients
4. Encoder-decoder = map sequence to different length sequence
5. Attention improves handling of long sequences

---

**Note\*\*:** Understanding these concepts is more important than memorizing.  
*Try to explain each idea in your own words!*

**All the best!**